ACADEMIC MINI-PROJECT COMPREHENSIVE EXPLANATION REPORT

Classical Ciphers & Symmetric Key Cryptography

An Interactive Encryption, Decryption, and Security Verification System

Document Purpose: This document is designed specifically for college project evaluations, seminars, and lab vivas. It provides a complete, from-scratch explanation of the system architecture, mathematical foundations of classical and modern ciphers, code implementation, step-by-step teacher demonstration workflow, and 15 curated viva questions with model answers.

Student / Candidate:	Sanket Jadhav
Course / Subject:	Cybersecurity / Cryptography & Network Security Lab
Core Technologies:	Python 3, Flask, Cryptography (PyCA), HTML5, CSS3, JavaScript
Key Security Primitive:	AES-256-GCM (Authenticated Encryption with Associated Data)
Project Repository:	digital_crypto_project (Zero-dependency virtual environment)
Evaluation Date:	Academic Year 2026

1. Executive Summary & Objectives

What is this project?
This project is a lightweight, full-stack educational cryptography laboratory. It visually and programmatically bridges the evolutionary gap between historical pencil-and-paper ciphers (Caesar, Monoalphabetic Substitution, Vigenère, and Rail Fence) and modern enterprise-grade symmetric cryptography (**AES-256-GCM**).

Key Objectives to state to your teacher:

• **1. Algorithmic Demonstration:** Provide hands-on implementation of both classical substitution (character identity change) and transposition (character position change) ciphers.

• **2. Production Cryptography:** Showcase how contemporary web and cloud systems protect data using **AES-256-GCM** with 256-bit keys.

• **3. The AEAD Concept (Confidentiality + Integrity):** Prove experimentally why encryption alone is insufficient without message authentication. Show how a 128-bit authentication tag detects bit-flipping attacks instantly.

• **4. Nonce Uniqueness:** Demonstrate why a fresh 96-bit random nonce is mandatory for every encryption call to prevent ciphertext determinism.

• **5. Zero Setup Friction:** Provide automated single-click deployment via run.bat without manual virtual environment configuration.

2. System Architecture & File Structure

The project follows a clean, single-backend, separation-of-concerns architecture designed intentionally without heavyweight frameworks to keep it simple, verifiable, and easy to explain during a viva.

File / Module	Role in Architecture	Key Cryptographic Functions
app.py	Flask web server and REST API controller	caesar_cipher, substitution_cipher, vigenere_cipher, rail_fence_cipher, aes_gcm_encrypt, aes_gcm_decrypt
templates/index.html	Responsive user interface	Workspace forms, dynamic key generator, AES metadata inspector, side-by-side comparison matrix
static/style.css	Cybersecurity UI styling	Dark blue & purple theme, responsive grid, code tags, glassmorphism cards
static/script.js	Client-side interactivity	Asynchronous fetch() API requests, real-time key generation, clipboard copying, input/output swapping
test_crypto.py	Automated verification suite	20 unit & integration tests covering round-trips, tampering detection, wrong key rejection, and API validation
run.bat	One-click execution script	Checks Python, creates .venv, installs dependencies, launches browser at 127.0.0.1:5000

3. Classical Ciphers Explained from Scratch

Classical ciphers are divided into two fundamental families: **Substitution Ciphers** (which swap character identities) and **Transposition Ciphers** (which rearrange character order).

3.1 Caesar Cipher (Monoalphabetic Shift Substitution)

• **Mechanism:** Shifts each letter in the message by a fixed integer key *k* positions along the alphabet.
• **Mathematical Formula:**
  Encryption: E(x) = (x + k) mod 26
  Decryption: D(c) = (c - k) mod 26
• **Security Vulnerability:** There are only 25 possible shifts in English. A modern computer or even a human with pencil and paper can break this in milliseconds using brute-force search across all 25 keys.

3.2 Monoalphabetic Substitution Cipher (Permutation)

• **Mechanism:** Maps each letter of the alphabet to a unique letter in a randomly shuffled 26-letter substitution alphabet.
• **Key Space:** 26! ≈ 4.03 × 10^26 possible permutations. Although brute force is mathematically infeasible, it is trivial to break.
• **Security Vulnerability:** *Frequency Analysis*. The cipher preserves natural language letter frequency distributions. In English, 'E' (~12.7%), 'T' (~9.1%), and 'A' (~8.2%) appear most frequently. By tallying letter frequencies in ciphertext, the mapping is easily discovered without knowing the key.

3.3 Vigenère Cipher (Polyalphabetic Substitution)

• **Mechanism:** Shifts letters using a repeating keyword string. If the key is 'KEY', the 1st letter shifts by K (10), 2nd by E (4), 3rd by Y (24), 4th by K (10), and so forth.
• **Historical Significance:** Known for over 300 years as *'le chiffre indéchiffrable'* (the unbreakable cipher) because it flattens simple single-letter frequency spikes.
• **Security Vulnerability:** Broken by Friedrich Kasiski (1863) using the **Kasiski Examination**. Repeated phrases in plaintext encrypted with the same key segment produce identical ciphertext fragments. Finding the greatest common divisor (GCD) of distances between repeats reveals the keyword length *L*. Once *L* is known, it becomes *L* separate Caesar ciphers.

3.4 Rail Fence Cipher (Zigzag Transposition)

• **Mechanism:** Plaintext is written diagonally along a zigzag pattern across *N* parallel rails (depth), then read off horizontally row by row.
• **Key:** Number of rails *N* (where $N \geq 2$).
• **Security Vulnerability:** Characters are never changed, only shuffled. All letter frequencies (unigrams, bigrams) remain 100% identical. An attacker can reconstruct the message by testing small rail depths (2 to 10) in seconds.

> ***Teacher Note / Viva Tip:*** *Make sure to emphasize to your teacher: 'Classical ciphers are strictly educational stepping stones. None of them provide confidentiality against modern cryptanalysis, and none of them provide any integrity checking.'*

4. Modern Symmetric Cryptography: AES-256-GCM

In modern systems (TLS 1.3, HTTPS, SSH, signal protocols, disk encryption), **AES-256-GCM** is the gold standard. This project implements AES-256-GCM using the audited cryptography library.

• **1. 256-bit Key Space:** A 256-bit key represents 2^256 (~1.15 x 10^77) possible keys. Even if all supercomputers on Earth checked 1 trillion keys per second, it would take billions of times the age of the universe to crack it by brute force. It is quantum-resistant against Grover's algorithm with an effective 128-bit post-quantum security margin.

• **2. Fresh 96-bit Nonce (IV):** A Nonce ('Number used ONCE') is a 12-byte random value. In our implementation, os.urandom(12) generates a fresh nonce for EVERY encryption. Even if you encrypt the word 'HELLO' 100 times with the same key, every single ciphertext is completely distinct. Nonce reuse in GCM is fatal and must be avoided.

• **3. 128-bit Authentication Tag (AEAD):** Encryption provides Confidentiality (secrecy). However, classical encryption does not prevent an attacker in the middle from flipping bits in transit. GCM computes a 16-byte Galois GHASH authentication tag. If even 1 bit of ciphertext is modified, tag verification fails and decryption aborts immediately.

Backend Implementation Walkthrough (from app.py):

5. Classical vs. Modern Comparison Matrix

Algorithm	Encryption Type	Key Method	Security Limitation	Modern Safe?
Caesar	Classical Shift	1 Integer (0-25)	Brute force in 25 tries; letter frequency.	**NO**
Substitution	Monoalphabetic	26-Letter alphabet	Vulnerable to letter frequency analysis.	**NO**
Vigenère	Polyalphabetic	Repeating keyword	Broken via Kasiski test and Friedman index.	**NO**
Rail Fence	Transposition	Rail count depth	Letters unaltered; anagramming & depth guessing.	**NO**
AES-256-GCM	Modern AEAD	256-bit key + 96-bit Nonce	Key must be kept secret; Nonce never reused.	**YES (Standard)**

6. Step-by-Step Teacher Demonstration Guide

Follow this exact 5-step walkthrough when presenting the project in front of your teacher or external examiner:

Step 1: Introduction (30 seconds)

State clearly: *'Good morning, respected teacher. My project is titled **Classical Ciphers and Symmetric Key Cryptography**. The goal is to demonstrate how cryptography evolved from ancient pencil-and-paper ciphers like Caesar and Vigenère to modern authenticated symmetric encryption like AES-256-GCM.'*

Step 2: Demonstrate Caesar & Substitution (1 minute)

Select **Caesar Cipher**, type Hello World!, set shift key to 3, and click **Encrypt** (output: Khoor Zruog!). Explain: *'Notice that every character shifts by 3. This is monoalphabetic substitution. Its weakness is that letter frequencies remain unchanged, making it vulnerable to statistical analysis.'*

Step 3: Demonstrate Rail Fence Transposition (1 minute)

Select **Rail Fence Cipher** with 3 rails and encrypt. Explain: *'Unlike Caesar which substitutes letters, Rail Fence transposes their positions along a zigzag path. The characters themselves never change.'*

Step 4: Demonstrate AES-256-GCM & Fresh Nonce (2 minutes)

Select **AES-256-GCM**. Click **Generate Secure Key** (64 hex chars = 256 bits). Enter a message and click **Encrypt**. Point to the **Cryptographic Metadata Inspector** below the output box showing the **96-bit Nonce**, **128-bit Auth Tag**, and **Ciphertext**. Click **Encrypt again** with the same message and key. Show your teacher that the ciphertext changes completely! Explain: *'Because we generate a fresh 96-bit cryptographically secure random nonce each time, identical plaintexts never yield the same ciphertext.'*

Step 5: The Showstopper: Tamper Detection (1 minute)

Click **Swap In/Out** so the ciphertext moves to the input box. Manually edit **just one character** of the Base64 ciphertext. Click **Decrypt**. Show your teacher the error message: Authentication tag mismatch! Explain: *'This demonstrates AEAD. Modern cryptography does not just hide data; it guarantees integrity. If a man-in-the-middle attacker tampers with even one bit, the Galois tag verification fails and the data is safely rejected.'*

7. Comprehensive Viva Questions & Answers

Q1: What is the fundamental difference between substitution and transposition?

Answer: Substitution replaces plaintext letters with other letters or symbols, changing their identity. Transposition rearranges the order of the letters without changing the letters themselves.

Q2: What is Kerckhoffs's Principle, and does AES follow it?

Answer: Kerckhoffs's Principle states that a cryptographic system must remain secure even if everything about the system (algorithms, source code, hardware) is public knowledge; only the key must remain secret. AES is completely public and standardized by NIST, yet mathematically secure.

Q3: Why does frequency analysis break monoalphabetic substitution so easily?

Answer: In any natural language, letter distributions are non-uniform (in English, 'E' is ~12.7%, 'T' is ~9.1%). Monoalphabetic substitution uses a fixed 1-to-1 mapping, so the ciphertext preserves these exact statistical peaks.

Q4: How was the Vigenère cipher broken if it flattens single-letter frequencies?

Answer: Via the Kasiski Examination and Friedman test. Repeating patterns in ciphertext reveal common factors of distance, which exposes the keyword length L. Once L is known, the ciphertext is split into L separate Caesar ciphers and solved.

Q5: What are the three standard key lengths of AES?

Answer: 128 bits (10 rounds), 192 bits (12 rounds), and 256 bits (14 rounds). Our project uses AES-256.

Q6: What is AEAD, and why is it superior to basic encryption modes like CBC or ECB?

Answer: AEAD stands for Authenticated Encryption with Associated Data. Basic modes (like ECB/CBC) only provide confidentiality and are vulnerable to padding oracle and bit-flipping attacks. AEAD provides simultaneous confidentiality and tamper-proof authentication.

Q7: What does GCM stand for in AES-GCM?

Answer: Galois/Counter Mode. It combines Counter mode (CTR) for high-speed streaming encryption with Galois field GHASH multiplication to compute a 128-bit authentication tag.

Q8: Why is Nonce reuse catastrophic in AES-GCM?

Answer: If the same Nonce and Key are used to encrypt two different messages, the XOR of the two plaintexts is revealed directly. Furthermore, the Galois hash key (H) can be recovered, allowing attackers to forge arbitrary authentication tags.

Q9: What happens when tampered ciphertext is submitted to AES-GCM decryption?

Answer: The receiver computes the GHASH over the received ciphertext. Even a 1-bit alteration causes a completely different hash output. The authentication tag verification fails, raising an InvalidTag exception, and the ciphertext is rejected without decoding.

Q10: What is the difference between symmetric and asymmetric cryptography?

Answer: Symmetric cryptography (e.g., AES) uses the same secret key for both encryption and decryption. Asymmetric cryptography (e.g., RSA, ECC) uses a mathematically linked key pair: a public key for encryption and a private key for decryption.

Q11: How is the symmetric key safely exchanged in real-world protocols?

Answer: Real-world protocols like TLS 1.3 use asymmetric cryptography (e.g., Elliptic Curve Diffie-Hellman Ephemeral - ECDHE) to securely agree upon a shared symmetric key, which is then used with AES-256-GCM for bulk data encryption.

Q12: Why did you use Python's 'cryptography' library instead of implementing AES manually?

Answer: Modern cryptography requires constant-time hardware-accelerated primitives (AES-NI) to prevent cache and timing side-channel attacks. Implementing AES manually in Python is vulnerable to timing attacks and is unsafe for production.

Q13: How many automated tests does your project have, and what do they verify?

Answer: We built a suite of 20 automated unit and integration tests in test_crypto.py that verify round-trip accuracy, non-alpha preservation, invalid key rejection, nonce uniqueness, and API tampering detection.

Q14: What is the key size used in your Caesar cipher and Rail Fence cipher?

Answer: Caesar uses an integer shift between 1 and 25. Rail Fence uses an integer rail count N >= 2.

Q15: What is the single biggest takeaway of this project?

Answer: Classical ciphers are valuable educational history, but real-world security requires modern authenticated primitives (like AES-256-GCM) that provide both mathematical confidentiality and cryptographic tamper resistance.